Laboratorio di Algoritmi e Strutture Dati

Ghirimoldi Luca 31974A

Progetto “Automi e segnali

Indice:

1. Struttura e organizzazione
2. Introduzione
3. Tipi utilizzati
4. Specifiche di progettazione
    a. Piano
    b. Automa
    c. Ostacolo
    d. coordinata
    e. Albero
        – Nodo
    f. Iteratore
    g. Coda

5. Specifiche di implementazione
6. Esempi di esecuzione

1.Struttura e organizzazione

utils/ package

├ coordinata.go : contiene la definizione e i metodi del tipo coordinata.

├ iteratore.go : contiene la definizione del tipo e i metodi del tipo Iteratore.

├ piano.go : contiene la definizione dei tipi utilizzati per modellare il problema con i relativi metodi di Piano.

├ pila.go : contiene la definizione del tipo pila FIFO, e i relativi metodi.

└ functions.go: contiene le operazioni definite nella traccia del progetto.

input/ Cartella contenente i file di input

output/ Cartella contenente i file di output rispettivi agli input

31974A_Ghirimoldi_Luca.go: file principale da eseguire con `go run 31974A_Ghirimoldi_Luca.go`

2.Introduzione

Il tipo Piano (utils.Piano) è stato progettato per rappresentare un insieme di automi e ostacoli distribuiti in un piano cartesiano.

Ogni Automa è caratterizzato dal suo nome e posizione nel Piano

Un ostacolo è un rettangolo nel piano, di punti che non è possibile attraversare, vengono memorizzate le coordinate dei vertici in basso a sinistra e in alto a destra.

3. Tipi utilizzati

I tipi utilizzati sono i seguenti:

- **Piano (utils.Piano):** struttura che rappresenta il piano, memorizza gli automi e gli ostacoli.
- **Automa (utils.Automa):** rappresenta un automa con il nome e la sua posizione.
- **Ostacolo (utils.Ostacolo)**: Struct che contiene p0 e p1 le due rispettive coordinate.
- **Coordinata (utils.coordinata):** rappresenta un punto del piano cartesiano dato dal valore dell'asse x e y
- **Albero (utils.Albero):** rappresenta un albero binario, tiene traccia della radice dell'albero
  - **Nodo (utils.Albero):** nodo dell'albero, memorizza il puntatore al nodo figlio di sinistra e destra e il puntatore all'Automa.
- **Iteratore (utils.Iteratore):** utilizzato per poter ottenere tutti gli Automi contenuti nei nodi figli di un Nodo specificato.
- **Coda (Utils.Coda):** Coda FIFO di un tipo generico (any)

4. Specifiche di Progettazione

Piano (utils.Piano) memorizza il puntatore alla radice dell'albero binario, e il puntatore alla slice di Ostacolo dove vengono memorizzati gli ostacoli, per far si che è possibile aggiungere un automa o un ostacolo all'interno della funzione esegui.

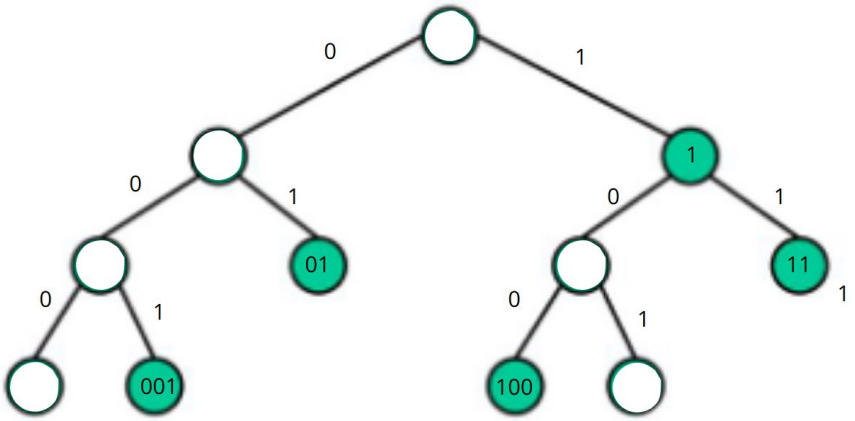
Il tipo Albero rappresenta la radice dell'albero binario.

I nodi di questo Albero, il tipo Nodo contiene il riferimento a 2 nodi "figli" di sinistra e destra, e il puntatore all'automa.

Ogni collegamento da un nodo al suo figlio è associato concettualmente un valore binario "0" per il nodo di sinistra e "1" per il nodo di destra.

La concatenazione concettuale dei valori binari lungo il percorso dalla radice fino ad un *nodo specifico* genera una sequenza binaria astratta, che è uguale al nome dell'automa contenuto in questo *nodo specifico*.

Ogni nodo non contiene necessariamente un Automa, infatti il nodo radice non conterrà mai alcun puntatore ad Automa.



(L'immagine aiuta a rendere l'idea, i nodi "bianchi" sono i nodi che non contengono il puntatore di un automa (nil), mentre quelli in verde sono i nodi che hanno un puntatore ad un automa con il nome specificato graficamente "all'interno" del nodo)

Per memorizzare gli automi distribuiti nel piano, ho utilizzato questa struttura ad albero binario, perché ogni automa ha un nome formato soltanto da “0” e “1”, ed è necessario poter ricercare tutti gli automi dato un prefisso.

La rappresentazione mediante albero mi è sembrata una scelta sensata per avere un implementazione che favorisce l’operazione di ricerca e inserimento.

Per effettuare la ricerca di un percorso libero tra un automa e un punto di richiamo ho implementato due strategie:

Se la posizione del punto dell’automa e del punto di richiamo condividono uno stesso valore dell’asse x o y, viene effettuato un controllo lineare ottimizzato se non è presente un ostacolo in quel segmento di piano.

Altrimenti, si considera il piano come se fosse un grafo non pesato, dove ogni punto è implicitamente un nodo, mentre i movimenti possibili (su,giù,destra,sinistra) rappresentano gli archi.

Ho scelto di implementare l’algoritmo di visita in ampiezza dei grafi (BFS) migliorato per poter trovare il percorso più breve che evita gli ostacoli, è migliorato in modo da visitare solo i nodi che si avvicinano al punto di richiamo, garantendo che il primo percorso trovato sia il più breve.

5. Specifiche di implementazione

Solution.go

In 31974A_Ghirimoldi_Luca.go è presente un tipo piano che è esattamente uguale a Utils.Piano, in modo da rispettare la signatura della funzione esegui(...) e newPiano()

La funzione `newPiano()` crea e restituisce un istanza del tipo `piano`, ottenuto tramite la chiamata alla funzione utils.Crea() che inizializza e restituisce un puntatore a utils.Piano.

Albero Binario

La memorizzazione degli automi tramite un albero binario realizzato in questo modo permette di avere operazioni di inserimento e ricerca di un automa di nome di lunghezza L, di $\Theta(L)$

É possibile effettuare operazioni di ricerca di tutti gli automi aventi lo stesso prefisso, utilizzando la funzione di ricerca per trovare il nodo “prefisso”, al quale viene effettuata una visita in profondità (DFS) per attraversare tutti i nodi figli del “prefisso” che contengono un Automa.
Per la ricerca lo spazio occupato è di $O(1)$.

Per l’inserimento la complessità dell’inserimento di un automa $\Theta(M)$, dove M è il numero di nuovi nodi creati

- Caso ottimale: è che il percorso esiste già -> $O(1)$ ($M = 1$)
- Caso peggiore: nome lungo L, $M = L$ non esiste nessun nodo, bisogna creare L nodi $\Theta(L)$
- Caso medio: Se solo gli ultimi K caratteri sono nuovi, si allocano $\Theta(K)$ nuovi nodi, con $K \leq L$

Iteratore

DFS è implementato tramite il tipo Iteratore che è un riferimento ad una Coda di puntatori a Nodo.

Alla chiamata del metodo .Iteratore() su un *Nodo, viene inizializzata la coda, e viene riempita tramite il metodo ricorsivo aggiungi(nodoRadice *Nodo) di Iteratore, aggiunge alla Coda, tutti i nodi figli di nodoRadice che contengono il puntatore di un Automa.

Effettua N chiamate ricorsive, dove N è il numero di tutti i nodi figli di nodoRadice.

Complessità temporale e spaziale di O(N).

La struttura iteratore è comoda per poter iterare su tutti gli automi contenuti in questa coda di Nodi, tramite i metodi .hasNext() e .Next(), il primo controlla se cè un altro nodo (automa) da ottenere, mentre il secondo restituisce un Automa. Entrambe le operazioni hanno tempo e spazio di O(1)

Coda

La struttura Coda[T] è una coda generica implementata tramite un slice dinamico per memorizzare gli elementi e un indice testa per tenere traccia della posizione del primo elemento.

Operazione	Complessità temporale	Complessità spaziale
Push()	O(1) ammortizzato	O(N)
Pop()	O(1)	O(N)
isEmpty()	O(1)	O(1)

La push è ammortizzato perchè occasionalmente l'operazione di append, raddoppia la capacità del slice.

La complessità spaziale è O(N) poichè lo slice può crescere fino a contenere tutti gli elementi.

Coordinate libere

Per controllare se una coordinata è libera tramite il metodo `*Piano.coordinateLibere(x int, y int) bool` bisogna controllare per tutti gli ostacoli se la coordinata(x,y) non è contenuta in ognuno di esso, eseguendo un ciclo che itera su tutti gli N ostacoli O(N).

Ricerca del percorso libero `*Piano.EsistePercorso(x int, y int, n string) bool`

Per effettuare la ricerca del percorso libero tra la posizione di un automa e un punto di richiamo (che per semplicità chiameremo distanza tra due punti) vengono effettuati alcuni controlli:

- Se la coordinata(x,y) è contenuta in un ostacolo non può esistere un percorso libero.
- Se non esiste un automa di nome n, non può esistere un percorso libero.
- Se non ci sono ostacoli, il percorso libero esiste, ed è pari alla distanza delle due coordinate.

  $D(A, B) = |xB - xA| + |yB - yA|$

Ammesso che la ricerca del percorso libero tra due punti è indifferente calcolarlo dall'automa al punto di richiamo o al contrario.

- Se le coordinate dei due punti condividono uno stesso valore dell'asse x o y, viene controllato che il segmento (orizzontale o verticale) tra i due punti non contiene ostacoli.

  Per farlo viene utilizzato un ciclo for, che itera i punti del piano contenuti nel segmento, partendo dal punto con coordinata minore, fino ad arrivare al punto con coordinata maggiore, controllando la presenza di ostacoli.

  Questo ciclo for ha un incremento di 2, dato che un ostacolo occupa almeno 2 coordinate del piano, perciò era inutile iterare su tutte le coordinate nel segmento.

- Se la posizione dell'automa e del punto di richiamo si trovano in coordinate con ascissa e ordinata differenti, viene implementato un algoritmo di *visita in ampiezza migliorato (Breadth-First Search, BFS)* dei grafi, come citato nelle specifiche di Progettazione.

  <u>Strutture dati utilizzate:</u>

  A. struct NodoPercorso che memorizza una coordinata e la sua distanza dalla coordinata iniziale (Automa).
  B. Mappa visitati: tiene traccia delle coordinate già esplorate, associando ad ognuna un valore booleano true, viene utilizzata una mappa per avere tempo di accesso O(1)
  C. Coda(queue): utilizzata per gestire l'esplorazione dei nodi in ordine di distanza crescente.

  L'algoritmo esplora il piano muovendosi dalla cordinata dell'automa, nelle 2 direzioni, che si avvicinano al punto del richiamo, in modo da evitare che si esplorano cordinate che si allontano dal punto di richiamo.

  per ogni nuova coordinata generata controlla:

  Se la coordinata corrisponde a quella di destinazione, il percorso libero è stato trovato e restituisce la distanza percorsa.

  Altrimenti verifica queste condizioni:

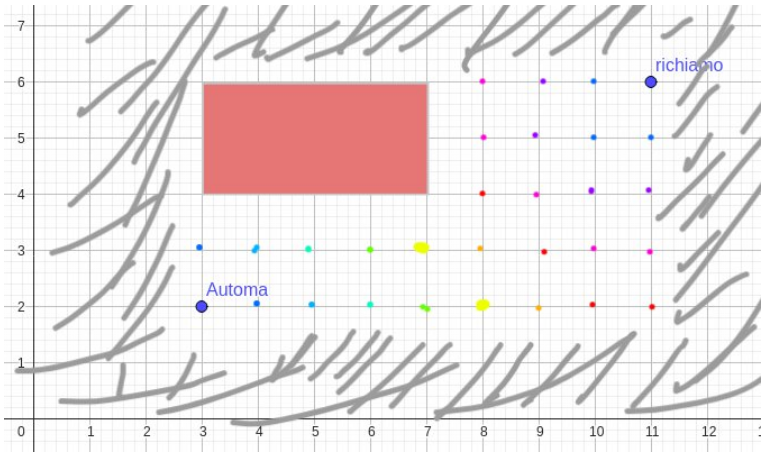
  1. Non è già stata visita (non è presente nella mappa *visitati*).
  2. È più vicina alla destinazione rispetto alla posizione attuale, evitando percorsi sub ottimali.
  3. Non contiene ostacoli.

  Al verificarsi di tutte e 3 le condizioni, la coordinata viene segnata come visitata (aggiunta alla mappa), e viene aggiunta alla coda.

  Grazie all'ottimizzazione di scelta delle direzioni consentite, si evita di muoversi in direzioni non necessarie, che avrebbero un percorso maggiore della distanza assoluta delle due coordinate.

  Permettendo di garantire che se viene trovato un percorso libero, esso è di distanza pari alla distanza delle due coordinate D(A,B) dove A e B sono le due coordinate del punto di richiamo, e dell'Automa.

Se BFS termina senza aver trovato il punto di destinazione, significa che non esiste un percorso libero tra i due punti, e l'algoritmo restituisce -1.



Rappresentazione grafica dei nodoPercorso aggiunti alla coda per arrivare da un automa ad un punto di richiamo, grazie all'ottimizzazione viene evitato di esplorare le coordinate in altre direzioni o che si allontanano dalla coordinata di richiamo.

(la seconda condizione serve per evitare di esplorare nodi non necessari (nella "zona grigia dell'immagine")

Piano.coordinateLibere(x,y) ha complessità di O(k) dove k è il numero degli ostacoli

Se non ci sono ostacoli ha complessità di O(1)
Cammino lungo una riga o una colonna: un ciclo itera al più O(d/2), con d la distanza tra le due coordinate.
Chiamata a bfs() se il punto non è allineato:
BFS esplora più volte gli stessi nodi, ma dato che vengono marcati come visitati nella mappa visitali, non vengono aggiunti alla coda dei nodi.

L'area della griglia rettangolare dove opera BFS, definita dalla coordinata di automa e del punto di richiamo, nell'immagine sopra la griglia va dalla coordinata (3,2) a (11,6) quindi sono 9 punti di "base" e 5 di "altezza", l'area è pari a (B)Base * (H) Altezza

Numero massimo di nodi esplorati O(B * H)

Anche se ogni nodo esplora solo due direzioni, la complessità temporale non cambia, perchè il numero di nodi esplorati è proporzionale alla dimensione della griglia.

Numero massimo di operazioni per ogni nodo: O(1), dato che esploriamo solo 2 direzioni O(2) = O(1).

Complessità spaziale:

La coda contiene al massimo O(B*H) *NodoPercorso, nel caso peggiore che non ci sono ostacoli nella griglia.
La mappa visitati memorizza uno stato per ogni nodo visitato O(B*H)
La variabile direzioni ha una dimensione fissa di 2 direzioni

6. Esempi di esecuzione:

Esempio 1

input/input1.txt file di input

output/output1.txt file di output

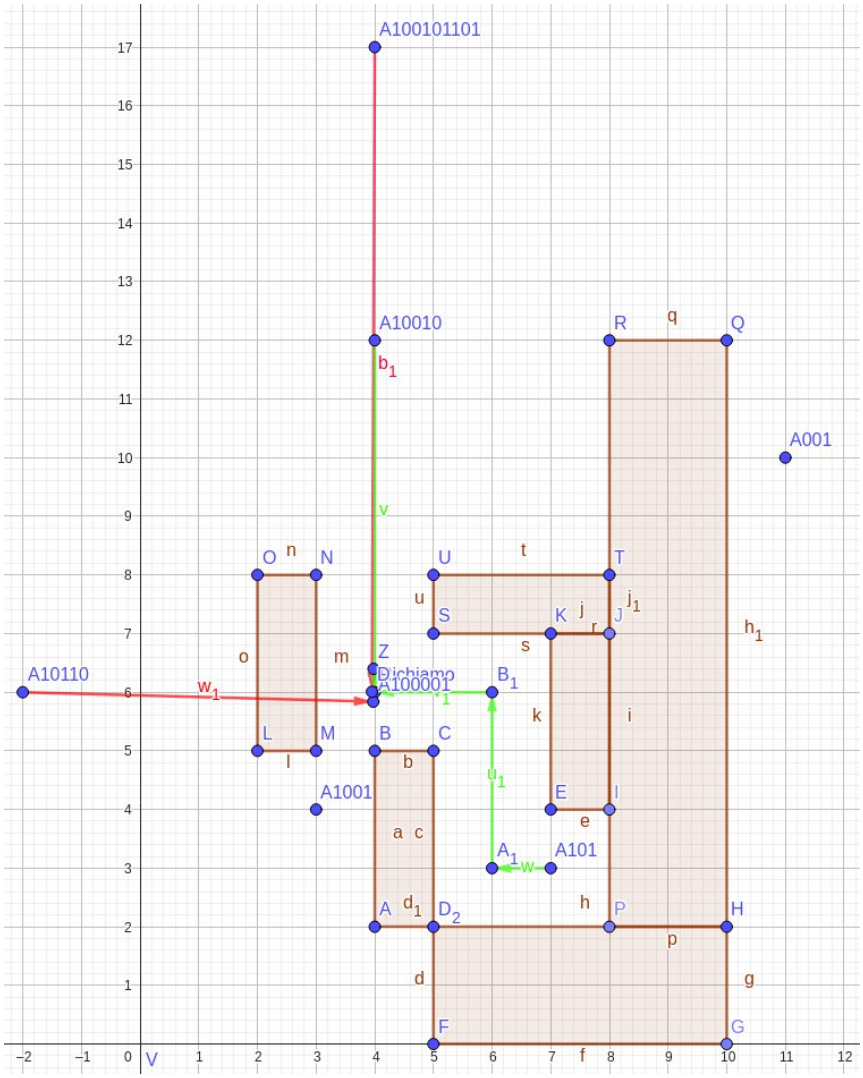
` go run 31974A_Ghirimoldi_Luca.go < input/input1.txt > output/actual1.txt `

I primi 3 comandi sono per verificare il comportamento corretto delle funzioni di posizioni, esistePercorso e Stato.

Successivamente crea un automa (101: 7,3) crea un ostacolo (4,2 : 5,5)

Prova a crea un automa su una posizione occupata dal primo ostacolo creato, e prova a creare u ostacolo sul primo automa creato (senza successo).

Successivamente crea 6 ostacoli, e 6 automi, ed effettua il richiamo degli automi che iniziano con “10” , risponderanno soltanto gli automi aventi lo stesso percorso libero minimo, l’immagine da una



rappresentazione grafica della situazione finale nel piano

(é possibile che sia presente qualche piccola imprecisione)

(Gli automi sono indicati con una A davanti al loro nome binario)

Le frecce verdi rappresentano il percorso che fanno gli automi richiamati.

Quelli rossi sono gli automi che vengono richiamati, ma non posso raggiungere il punto di richiamo o perchè troppo lontani (100101101) o perchè hanno il percorso occupato da ostacoli (10110)

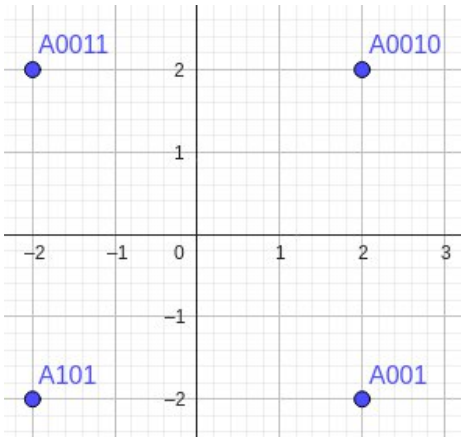
Esempio 2

input/input2.txt file di input

output/output2.txt file di output

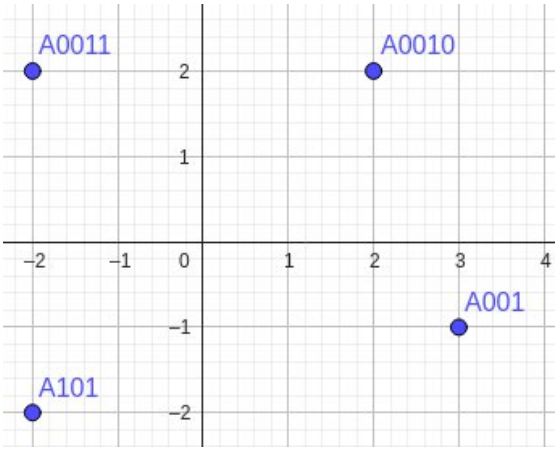
` go run 31974A_Ghirimoldi_Luca.go < input/input2.txt > output/actual2.txt `

Inizializzato il piano così

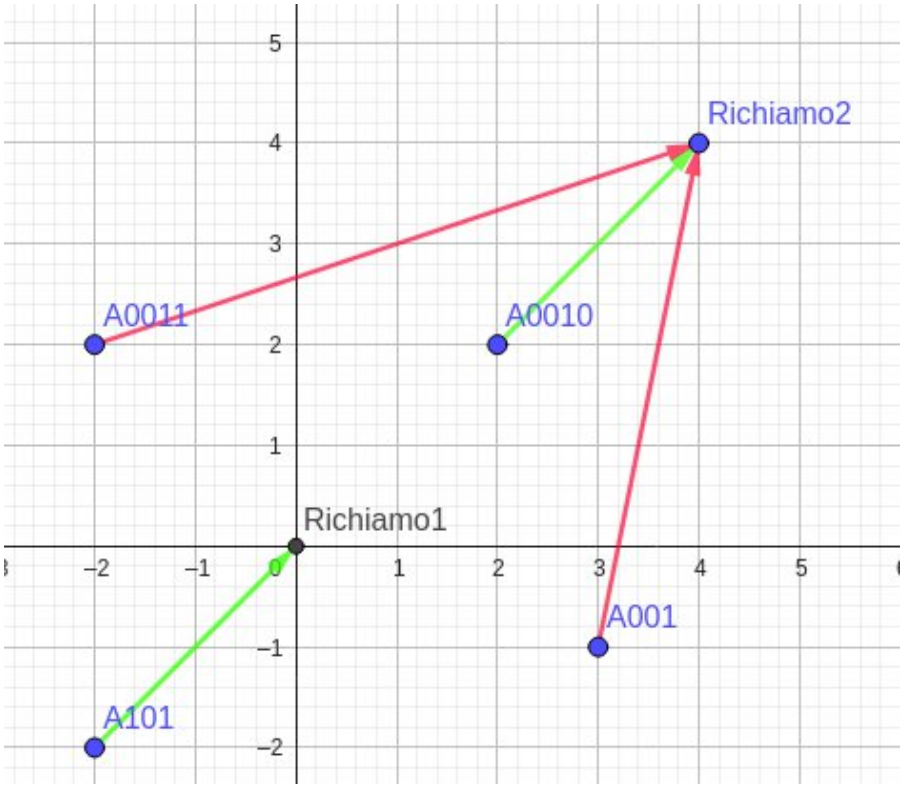


poi viene creato un automa di nome 001 (già esistente: e quindi viene spostato nella nuova posizione.

=======>



Vengono fatti 2 richiami nel punto (0,0) per gli automi con prefisso “101” e nel punto (4,4) per gli automi di prefisso “00”



Le frecce indicano dove verrano richiamati gli automi con successo (se il colore è verde)

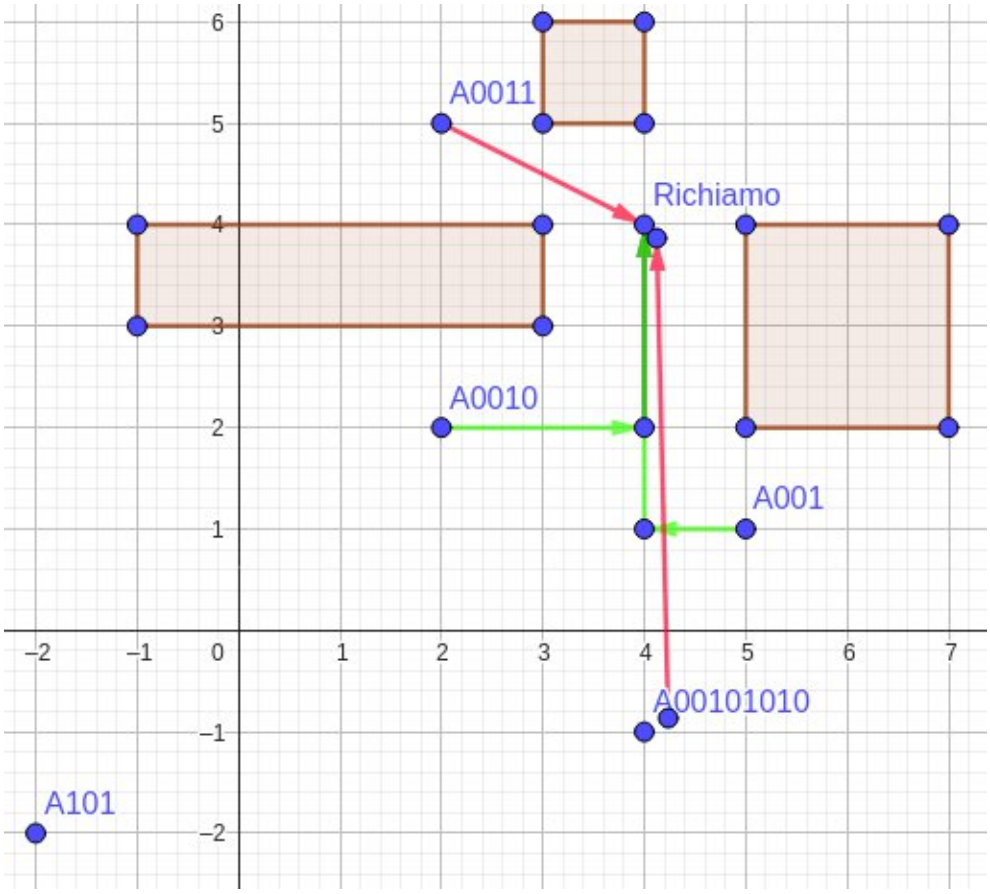
Dato che non sono presenti ostacoli, tutte le coordinate sono raggiungibili da un percorso libero, ma solo l’automa con distanza minima risponde al richiamo.

Esempio 3

input/input3.txt file di input

output/output3.txt file di output

` go run 31974A_Ghirimoldi_Luca.go < input/input3.txt > output/actual3.txt `



Richiamo di “00”

0011 non potrà raggiungere il richiamo perchè non cè un percorso libero.

0010 e 001 potranno raggiungere il richiamo

101 non è prefisso del richiamo di “00”

00101010 è prefisso ma è troppo lontano